



SAPIENZA
UNIVERSITÀ DI ROMA

MIP Modelling

Integer linear programming models
for making optimal decisions

Fabio Furini

Department of Computer, Control and Management Engineering
Sapienza University of Rome

A.Y. 2026–2027

Contents of the course

Part I — Modelling

- What a MIP model is
- Logic and binary variables
- The fourteen links between variables
- Relaxations, duality and bounds
- Constructive heuristics
- From the model to Python/Gurobi

Part II — The problems

- Fifteen numerical models (EX 1–15)
- Assignment and scheduling (7 problems)
- Location and covering (4 problems)

- Production planning (3 problems)
- Mixed problems (9 problems)

The course

- Organisation and exam format
- Notebooks in Colab

Material

Site: fabiofurini.github.io/mip-modelling

Code: github.com/fabiofurini/mip-modelling

The method of the course, in one line

For every problem

model $\rightarrow$ proof of the links $\rightarrow$ instance $\rightarrow$ heuristic (primal bound) $\rightarrow$ dual of the relaxation (dual bound) $\rightarrow$ solution with the solver $\rightarrow$ additional modelling questions

- First the model is **built**: variables, objective, constraints. Where two families of variables are linked, one explains what the implication is and how the constraint (or optimality) imposes it.
- Then the optimum is **enclosed** between two numbers: a heuristic gives the bound on one side, a dual solution of the LP relaxation gives the bound on the other.
- Finally the model is **solved**, with Gurobi from Python.

Why the sandwich matters

It is the technique used in practice when the instance is too large to be solved to proven optimality: the two bounds say what the solution in hand is worth.

Notation

- **LP** linear programming; **ILP** integer; **BIP** binary; **MILP** mixed integer.
- Numbered objects with indices over enumerated sets: $j \in \{1, 2, \dots, n\}$ — the notation of the text is already that of the model; rational data ($\mathbb{Q}$), integer counts ($\mathbb{Z}_{\geq 1}$).
- The **bar** marks the values of a *feasible* solution ($\bar{x}_j$), the **tilde** those of an *optimal* one ($\tilde{x}_j$).
- Dual variables: π for capacity constraints, μ for equalities; α, β, γ when more families are needed. Never y for the duals.
- Bounds and values: UB , LB , $z(LP)$ (relaxation without the bounds), $z(LP^+)$ (with the bounds), $z(MILP)$.
- In every model: first the **data** (with their type), then the **variables** (with their count), then objective and constraints.

What a MIP model is

Data, variables, objective, constraints

- The **data** are given: they are not decided. In the model they appear as symbols with their type ($c_j \in \mathbb{Q}_{>0}$, $n \in \mathbb{Z}_{\geq 1}$).
- The **variables** are the decisions. They are declared one family at a time, with the count: "we introduce the following n k binary variables".
- The **objective** is a linear function of the variables.
- The **constraints** are grouped by family, and every family is described in words, with the number of constraints it contains.

The rule that is not negotiable

Every row of the model must be **justified**. A constraint nobody can explain is a constraint nobody can fix.

Four classes, one language

Class	Variables	Example in the course
LP	all continuous	(the Operations Research Lab)
ILP	all integer	EX 13, funds in lots
BIP	all binary	EX 1, the van
MILP	mixed	EX 5, vehicles with a minimum quantity

The point

The class changes, the language does not: data, variables, objective, constraints. But what can be *said* about the solution changes radically — and so does the cost of finding it.

Why integrality matters

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s.t.} & 2x_1 + 2x_2 \leq 3 \\ & x_1, x_2 \in \{0, 1\}\end{array}$$

- The LP relaxation replaces $x \in \{0, 1\}$ with $0 \leq x \leq 1$ and is worth $z(LP^+) = 3/2$.
- That value is attained by **infinitely many** optimal solutions — the whole segment $x_1 + x_2 = 3/2$ inside the square.
- Rounding is not an approximation: it is **a mistake**. The integer optimum is 1, not 2, and no rounding finds it by chance.

To remember

The value of the relaxation is not "almost" the optimum of the MILP: it is a *bound*. The difference $z(MILP) - z(LP)$ has a name — the price of integrality — and in some problems of the course it is 30%.

The sandwich of the bounds

The chain that holds the whole course together

$$LB \leq z(LP) \leq z(MILP) \leq UB \quad (\text{minimisation problem})$$

- UB comes from a **feasible solution**: any feasible solution is a bound from its own side.
- LB comes from a **feasible dual solution** of the relaxation: by weak duality, *always*, with no assumptions.
- In a **maximisation** problem the two roles swap: the heuristic gives a lower bound, the dual an upper bound.

What the solver does with them

Branch-and-bound does exactly this, at every node: a bound from the relaxation and one from a solution found. When the two meet, it is done.

Logic and binary variables

From a sentence to a constraint

- A binary variable is a **proposition**: $x_j = 1$ means "project j is funded".
- The logical connectives (AND , OR , NOT) are translated into linear constraints, through conjunctive normal form.
- Every clause becomes *one* linear constraint.

Sentence	Constraint
at most one of $x_1, \dots, x_n$	$\sum_j x_j \leq 1$
at least one	$\sum_j x_j \geq 1$
exactly one	$\sum_j x_j = 1$
if x_1 then x_2	$x_1 - x_2 \leq 0$
x_1 and x_2 not together	$x_1 + x_2 \leq 1$

Check the translation, do not trust it

Mutual exclusion: what it does *not* say

The constraint $x_1 + x_2 \leq 1$ says "not both". It does **not** say "at least one": $x_1 = x_2 = 0$ satisfies it. The converses must be refuted explicitly.

- The safe way to check a translation is to **enumerate**: with few binaries all assignments are listed and the feasible set is checked against the sentence.
- In the course every translation is checked this way, by a script, with an assert.

How much six implications cut

On the ten projects of exercise 2.1: the $2^{10} = 1024$ assignments drop to 234 once all six implications are imposed — less than a quarter. None of them, on its own, cuts more than half.

The fourteen links between variables

The map of the techniques

1. Activation
2. Fixed cost
3. Minimum lot
4. Integer counts
5. Maximum variable
6. Min-max and max-min
7. Absolute value
8. Big-M
9. Precedences
10. If and only if
11. Counting types
12. Alldiff and binary expansion
13. Soft constraints
14. Piecewise linear functions

Every technique in five steps

(a) the link in words; (b) the linear constraints; (c) the proof in both directions, or the exchange argument; (d) how strong the LP relaxation is; (e) the gurobipy line.

Activation and fixed cost

The link

"The quantity q_j can be positive only if activity j is on."

$$q_j \leq c_j y_j, \quad \forall j \in \{1, 2, \dots, m\} \quad (m \text{ constraints}).$$

- **Direction imposed by the constraint:** if $y_j = 0$ then $q_j \leq 0$, that is $q_j = 0$.
- **Direction that follows from optimality:** if $q_j > 0$ then $y_j = 1$ — but only because y_j has a **strictly positive** cost in the objective. With zero cost the correct conclusion is weaker: *there exists* an optimum with $y_j = 0$.

The mistake not to make

Saying "in every optimum" when the coefficient is only ≥ 0 . If the fixed cost can be zero, the statement must be weakened.

Minimum lot: two constraints, one semi-continuous variable

$$\begin{aligned}\ell y_j &\leq q_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ constraints}), \\ q_j &\leq c_j y_j, & \forall j \in \{1, 2, \dots, m\} & \quad (m \text{ constraints}).\end{aligned}$$

y_j	what the two constraints impose	admissible q_j
0	$q_j \geq 0$ and $q_j \leq 0$	only $q_j = 0$
1	$q_j \geq \ell$ and $q_j \leq c_j$	$\ell \leq q_j \leq c_j$

- Neither is enough on its own: without the second one produces with $y_j = 0$; without the first the minimum is vacuous.
- One needs $\ell \leq c_j$: otherwise $y_j = 1$ is infeasible and the variable is *forced* to zero — a data error the solver does not report.

Big-M: the smallest the data justify

The rule

M is computed **from the data**, and its value is written right after the model: " $M = \sum_j t_j$ is enough".

- A huge M "to be safe" does not make the model more correct: it only **weakens the LP relaxation**.
- With M too large the relaxation leaves y_j near zero while x_j is positive: the bound weakens and branch-and-bound has to work harder.
- In EX 5 the big-Ms are (66, 80, 50, 44, 36): one per type, all limited by the hours of labour. They are not constants of the model, they are **functions of the data of the instance**.

To remember

Data change $\Rightarrow$ big-Ms change. In EX 12, raising the leather from 120 000 to 200 000 grams, the big-Ms go from (141, 200, 171) to (235, 333, 285).

Maximum variable, min-max, absolute value

$$\begin{aligned} z &\geq t_j x_j, \quad \forall j \in \{1, 2, \dots, n\} && (n \text{ constraints}), \\ z &\geq 0. \end{aligned}$$

- The constraints impose $z \geq \max_j t_j x_j$; **optimality** makes it an equality, because z is minimised.
- The absolute value $|u - v|$ in a *minimisation* costs one variable $d \geq 0$ and two constraints, $d \geq u - v$ and $d \geq v - u$.
- In a *maximisation* the same trick fails: a binary and a big-M are needed, because the disjunction has to be imposed, not obtained for free.

"Balanced" is not an objective: it is a word

Minimising the *maximum* of the loads and minimising their *difference* give the same optimal solutions but different values: in EX 11, 9 against 0. Reporting one for the other is a mistake.

If and only if: when the link must hold both ways

- $x_i \leq y$ says "if there is activity then the indicator is on", **not** the converse.
- If a constraint then *counts* the indicators — say $\sum_i y_i \geq 2$ — the missing direction makes that constraint **vacuous**: it is enough to switch y on without doing anything.

The case of EX 15, the music school timetable

With the link $x_{dti} \leq y_{di}$ alone the solver returns a timetable where Thursday has a single instrument — three hours of guitar — and it is *feasible*. What is also needed is

$$y_{di} \leq \sum_{t \in T} x_{dti}.$$

The rule

When a constraint counts indicators, the link must hold in both directions.

Relaxations, duality and bounds

What a relaxation is

- A relaxation enlarges the feasible region: every solution of the MILP stays feasible, so its optimum is a bound.
- The regions include each other the right way round:

$$X_{MILP} \subseteq X_{LP^+} \subseteq X_{LP}.$$

- $z(LP)$ is the relaxation **without the bounds** ($x_j \geq 0$ only): it is the one whose dual the course writes by hand.
- $z(LP^+)$ is the relaxation **with the bounds** ($0 \leq x_j \leq 1$): it is what the solver does with `relax()`, and it is tighter.

There is no "dual of the MILP"

The dual one writes is always that of the *relaxation* named. A MILP has no linear dual, and the gap $z(MILP) - z(LP)$ is precisely what is missing.

The primal/dual conversion table

In the primal (min)	In the dual (max)
constraint i of direction $\geq$	variable $\pi_i \geq 0$
constraint i of direction $\leq$	variable $\pi_i \leq 0$
equality constraint i	free variable π_i
variable $x_j \geq 0$	constraint j of direction $\leq c_j$
free variable x_j	equality constraint $j, = c_j$

- Dual objective $\max \sum_i b_i \pi_i$; dual constraint j : $\sum_i a_{ij} \pi_i \leq c_j$.
- If the primal is a **maximisation**, all directions flip.

How to read a dual constraint

"The value I attach to the resources consumed by activity j cannot exceed its cost." With this reading every recipe has an economic meaning — and that is what makes it memorable.

Weak and strong duality

Weak duality — this is the one we need

$\sum_i b_i \bar{\pi}_i \leq \sum_j c_j \bar{x}_j$, *always*, with no assumptions. It gives a bound from **any** feasible dual solution, even one built by hand and far from optimal.

Strong duality — this one is a check

If the LP has a finite optimum, $z(D(LP)) = z(LP)$. The optimum of the dual written by hand *must* match $z(LP)$: if it does not, the dual is written wrong. In the course an assert checks it.

$$\sum_{i=1}^m b_i \bar{\pi}_i \leq z(LP) \leq z(MILP)$$

A dual bound of the LP is a bound for the MILP too, from the same side.

Three recipes for building a dual solution by hand

1. **Zero everything but one family.** The remaining ones are pushed to the largest value compatible with the dual constraints. In 7.1: $\bar{\pi} = 0$ and $\bar{\mu}_j = \min_m c_{jm}$ — "every job costs at least its cheapest machine".
2. **Constructive heuristic on the constraints.** The primal constraints are scanned one by one, the corresponding dual variable is raised until the first opposing dual constraint is tight, and the residuals are updated.
3. **The best ratio.** With a single capacity constraint in a maximisation, $\bar{v} = \max_j p_j/w_j$ is feasible and gives the bound $b \bar{v}$: "all the capacity used at the best efficiency".

The only thing that makes the bound valid

The solution built must be **checked feasible**, on every dual constraint, one by one. An unchecked dual value is not a bound.

A certificate that does not hold: EX 13

$$\begin{array}{ll}\max & 2x_1 + 3x_2 \\ \text{s.t.} & 12x_1 + 20x_2 \leq 100 \\ & -x_1 + x_2 \leq 0 \\ & x_1, x_2 \in \mathbb{Z}_{\geq 0}\end{array}$$

Dual: $\min 100\alpha$ with $12\alpha - \beta \geq 2$ and $20\alpha + \beta \geq 3$.

A draft proposed $\bar{\alpha} = 5/32$, $\bar{\beta} = 1/8$:

$$12 \cdot \frac{5}{32} - \frac{1}{8} = \frac{56}{32} = \frac{7}{4} < 2.$$

Violated by 1/4: not a feasible solution, and $100 \cdot 5/32$ is not a bound.

The correct recipe

$\bar{\beta} = 0$ and $\bar{\alpha} = \max_j p_j/c_j = 1/6$ — "a million is worth what it returns in the better fund" — giving $UB = 50/3 \approx 16.67$, against $z(MILP) = 16$: the difference $2/3$ is the price of integrality.

LP duals are not the marginal prices of the MILP

The most expensive mistake of the chapter

In an LP the dual π_i is the marginal price: one more unit of resource i is worth π_i . In a MILP it is not. The integer optimum is a step function of the availability: between steps one more unit is worth nothing, on a step it is worth a lot.

- The π_i values the solver returns are those of the *relaxation*, not of the MILP.
- To know what one more unit of a resource is really worth in a MILP there is only one way: **solve it again** with the data changed.

Constructive heuristics

What a constructive heuristic is

- It builds a solution **one piece at a time**, never backtracking, following a declared rule.
- It serves two purposes: giving a bound on the primal side, and giving the student a way of *understanding* the problem before delegating it to the solver.
- It must be run **step by step**, showing the residual availabilities: that is the part graded at the exam.

Rule	Where it is used
next-fit, first-fit, best-fit	packing into containers
the least loaded (LPT)	balancing, EX 11
the best ratio	covering, EX 14
the cheapest completion	lot sizing

When a heuristic fails, and what it gives

- A constructive heuristic can **get stuck**: in EX 9 it places five queens and then finds no free squares. The bound stays valid, but it is weak — 5 against 8.
- It can also **be off by very little**: in EX 5 the gap is 0.2%.
- What it *always* gives is a **feasible** solution: and that must be checked, not assumed.

What it does not give

It does not give the optimum, and it does not say how far it is. Only the bound from the other side says that: which is why the two halves of the course belong together.

From the model to Python/Gurobi

The model, one family per block

- One solver only: **Gurobi from Python**. One `addConstrs` per family, named after the label of the model.
- The name is not decoration: it is what lets one read the duals and match code and notes line by line.

The protocol of the course, from start to end

1. the model is written and solved;
2. Status is read — **always before** `ObjVal`, which on an unsolved model raises an error;
3. the heuristic is built and its feasibility *checked*;
4. the dual is written, a solution is built by hand and checked;
5. LB , $z(LP)$, $z(LP^+)$, $z(MILP)$, UB are compared in a table.

Reproducibility

```
python3 python/run_all.py regenerates data, figures and notebooks;  
python3 python/check_numbers.py checks every number quoted in the notes.
```

Tolerances

- A solver works in floating point: " $x_j = 1$ " is read `x[j].X > 0.5`, never `== 1`.
- A "satisfied" constraint is satisfied within a tolerance. In bound comparisons the course uses 10^{-6} .
- Fractions are reported as fractions (25/4, 50/3), not as truncated decimals: that is how exactness is not lost between code and text.

The "no invented number" rule

Every number appearing in the notes, in the slides and on the site comes from a script, and is checked by an assert. If a number changes, the check fails and the CI does not publish.

The problems

How Part II is built

- **Fifteen numerical models** (EX 1–15): explicit data, few variables, one technique each. They are read to get one's bearings.
- **Four families** of general problems, with symbolic model, instance, heuristic, dual and additional questions:
 - assignment and scheduling — 7 problems;
 - location and covering — 4 problems;
 - production planning — 3 problems;
 - mixed problems — 9 problems.

The rule for writing models

The **general** model is always symbolic (n, m, c_j); the **instance** model is always written **out in full**, one column per variable. Never a model halfway between the two.

EX 1 — the eight-seat van

$$\begin{array}{llll} \max & 30x_1 & +50x_2 & +80x_3 & +70x_4 \\ \text{s.t.} & 2x_1 & +3x_2 & +4x_3 & +5x_4 \leq 8 \\ & x_1 & +x_2 & +x_3 & +x_4 \leq 2 \\ & & x_2 & & -x_4 \leq 0 \\ & x_1, & x_2, & x_3, & x_4 \in \{0, 1\} \end{array}$$

LB (heuristic)	110
UB (dual by hand)	160
$z(LP)$	160
$z(LP^+)$	140
$z(MILP)$	120

- The dual by hand ($\bar{\beta} = \bar{\gamma} = 0$, $\bar{\alpha} = \max_j p_j/a_j = 20$) is *optimal* for the relaxation without the bounds: "a seat is worth as much as the group that pays best for it".
- Heuristic gap 8.3%; certified gap 41.7%.

Assignment and scheduling — seven problems

	Problem	What it teaches
7.1	minimum-cost assignment	the base family: no link to prove
7.2	machines with a fixed cost	aggregated activation, and its weak relaxation
7.3	job selection	not all jobs have to be done
7.4	parallel jobs	several machines for the same job
7.5	classes with setup	the setup is paid per class, not per job
7.6	classes with a bonus	an "if and only if" with two link constraints
7.7	total tardiness	sequencing with big-M, and the right big-M

Location and covering — four problems

	Problem	What it teaches
8.1	capacitated location	activation with a capacity
8.2	p -median	the number of sites is fixed, not costed
8.3	covering with interference	set covering plus mutual exclusion
8.4	hub with a maximum cost	the maximum variable, and four relaxations

Aggregated against disaggregated (8.4)

The same link can be written aggregated or disaggregated. *Adding* the disaggregated links tightens the relaxation; *replacing* the aggregated one loses the capacity and makes the optimum collapse from 19 to 10. The four relaxations are 1015/78, 79/6, 11, 10.

Production planning and mixed problems

Production — 3 problems

- 9.1 lot sizing with a fixed cost
- 9.2 production and workforce
- 9.3 vehicles with a minimum lot

Here the variables do not say *whether*, they say *how much*: inventory balance, workforce, hiring.

Mixed problems — 9 problems

- 10.1 prizes with two modes
- 10.2 combinatorial auction
- 10.3 diet with a minimum lot
- 10.4 trees and boxes of lights
- 10.5 shipments in boxes
- 10.6 children among summer camps
- 10.7 branches between two companies
- 10.8 tracks among CDs
- 10.9 books among shelves

Where the LP relaxation stops helping

In the mixed problems the relaxation is often worth zero. The bound from below has to be found with **combinatorial** arguments — parity, counting the containers — not with the dual.

Organisation of the course

The format of every exercise — and of the exam

1. **Model.** Variables (with their count), objective, constraints with their description; and if two families are linked, the **proof of the link in both directions**.
2. **Instance.** The model written out in full on a small numerical instance.
3. **Heuristic.** A constructive algorithm, run step by step, giving the bound on the primal side.
4. **Dual.** The dual of the LP relaxation — general and on the instance — and a feasible dual solution built by hand.

The thread through the course

$LB \leq z(MILP) \leq UB$: a model is not merely written, it is **tightened from both sides** before being handed to the solver.

Marking criteria

Dimension	Weight
Correctness of the model (variables, objective, constraints)	35%
Proof of the link between the variables (both directions)	25%
Constructive heuristic and its correct execution	20%
Dual of the relaxation and a feasible dual solution	20%

Typical discussion questions

Is the link constraint aggregated or disaggregated, and what difference does it make to the relaxation? Is the opposite direction of the implication imposed by the constraint or does it follow from optimality? What is the smallest big-M the data justify? Does the heuristic find the optimum, and how does one know without the solver?

The eight most common mistakes

1. Writing an implication whose conclusion is true anyway (vacuous contrapositive).
2. Proving only one direction of an implication "imposed by the constraint": both are always needed.
3. Confusing "imposed by the constraint" with "follows from optimality": the second needs the exchange argument, "one can see it pays off" is not enough.
4. Concluding "in every optimum" when the coefficient is only ≥ 0 : the correct conclusion is "there exists an optimum in which...".
5. Using a huge big-M "to be safe": it weakens the relaxation for nothing.
6. Forgetting that $A + B \leq 1$ does **not** imply that at least one of the two is 1.
7. Getting the direction or the sign of a dual variable wrong with respect to the primal constraint.
8. Building a dual solution and not checking its feasibility on every constraint.

The material of the course

- **Site** fabiofurini.github.io/mip-modelling — the notes online, with the code embedded in every page.
- **Notebooks in Colab**: one per model, opened from the badge at the top of each page and running in the browser. The licence shipped with `gurobipy` is enough for every instance of the course.
- **Code** at github.com/fabiofurini/mip-modelling, under the MIT licence; text, figures and data under CC BY 4.0.
- Italian version: fabiofurini.github.io/modellazione-mip.

Enjoy the course.